

22 ответа на вопросы по SQL из собеседований

 [Изи гайд по аналитике](#)

 [SQL для подготовки к собеседованиям на Stepik](#)

Содержание

1	Введение	2
2	Основы и базовый синтаксис	3
2.1	Разница между WHERE и HAVING?	3
2.2	Какой порядок выполнения SQL запроса?	3
2.3	В чем отличие между DELETE, TRUNCATE, ALTER TABLE	3
2.4	Что такое PRIMARY и FOREIGN ключи?	3
2.5	Как убрать дубликаты из данных?	4
2.6	Как работает алгебра логики в SQL?	4
3	Операторы объединения данных	4
3.1	Какие виды джойнов вы знаете?	4
3.2	В чем отличие между INNER JOIN и LEFT JOIN?	5
3.3	В чем разница между UNION и UNION ALL?	6
4	Оконные функции	6
4.1	Синтаксис оконных функций	6
4.2	Какие виды оконных функций вы знаете?	6
4.3	В чем разница между RANK() и DENSE_RANK()?	7
4.4	Разница между LAG() и LEAD()?	7
4.5	Фреймы: ROWS и RANGE. Что это такое и как они работают?	8
5	Что такое подзапрос SQL?	8
5.1	Подзапрос SQL	8
5.2	Как используется подзапрос?	9
6	Оптимизация кода	9
6.1	Как оптимизировать запрос?	9
6.2	В чем отличие EXPLAIN от EXPLAIN ANALYZE?	10
7	Общие вопросы про базы данных	11
7.1	Какие отличия между объектами БД Table, View, Materialized View, External Table?	11
7.2	Что такое транзакция?	11
7.3	Принципы ACID	12
7.4	Индексы БД. Что это такое?	12

1 Введение

Этот файл - краткое содержание нашего курса SQL для подготовки к собеседованиям <https://stepik.org/course/254086/promo>. Советуем использовать этот файл как шпаргалку перед тем, как идти на собеседование, чтобы повторить основные вопросы и структурировать ответы у себя в голове.

2 Основы и базовый синтаксис

2.1 Разница между WHERE и HAVING?

Ключевое отличие в порядке выполнения запроса. **WHERE** фильтрует отдельные строки **до** группировки, а **HAVING** фильтрует результаты агрегации **после** группировки. Проще говоря: **WHERE** работает с полями, а **HAVING** — с агрегатными функциями (как SUM(), COUNT(), AVG()).

2.2 Какой порядок выполнения SQL запроса?

№	Команда	Что делает операция
1	FROM	определяет, из каких таблиц мы выбираем данные
2	WHERE	фильтрует строки в соответствии с определенными условиями
3	GROUP BY	группирует строки на основе определенных столбцов
4	HAVING	фильтрует группы строк, созданных оператором GROUP BY, в соответствии с заданными условиями
5	SELECT	выбирает столбцы, которые должны быть включены в результаты запроса
6	ORDER BY	сортирует результаты по определенным столбцам
7	LIMIT	ограничивает количество выбранных строк или определяет смещение для выборки строк

2.3 В чем отличие между DELETE, TRUNCATE, ALTER TABLE

- **DELETE** — удаляет данные (строочки)
- **TRUNCATE** — удаляет все данные сразу
- **ALTER TABLE** — меняет структуру таблицы (столбцы, ограничения)

Критерий	DELETE	TRUNCATE	ALTER TABLE
Тип команды	DML	DDL	DDL
Что делает?	Удаляет строки (можно с WHERE)	Быстро удаляет ВСЕ строки	Меняет схему таблицы (столбцы, типы)
Можно откатить?	Да (в транзакции)	Зависит от СУБД (чаще — да)	Зависит от СУБД и операции
Скорость	Медленно	Очень быстро	Зависит
Сбрасывает счётчик ID?	Нет	Да	Нет
Срабатывают триггеры?	Да	Нет	Нет

2.4 Что такое PRIMARY и FOREIGN ключи?

Primary Key (он же первичный ключ) - это столбец (атрибут), уникально идентифицирующий каждую строку в таблице.

Foreign Key (он же внешний ключ) - это столбец (атрибут), который ссылается на Primary Key другой таблицы. Он обеспечивает связь между таблицами и целостность данных.

2.5 Как убрать дубликаты из данных?

Метод	Когда использовать	Плюсы	Минусы
DISTINCT	Нужно быстро получить уникальные комбинации значений определенных столбцов	Простота и читаемость	Низкая производительность
GROUP BY	Нужны уникальные значения и агрегация по ним (посчитать сумму, количество)	Универсальность, возможность использовать агрегатные функции	Избыточен, если агрегация не нужна
ROW_NUMBER	Нужно удалить дубликаты, оставив конкретную строку (последнюю, первую, с максимальным ID)	Максимальная гибкость в выборе строки	Сложный синтаксис

2.6 Как работает алгебра логики в SQL?

Оператор	Назначение	Пример	Результат
AND	Истина, если оба условия истинны	age > 18 AND city = 'Москва'	Истина только если оба верны
OR	Истина, если хотя бы одно условие истинно	city = 'Москва' OR city = 'СПб'	Истина, если совпадает хотя бы одно
NOT	Инвертирует логическое значение	NOT (city = 'Москва')	Истина, если статус не 'Москва', фактически работает как != (или не равно)
IN	Проверяет, входит ли значение в список	country IN ('Россия', 'Беларусь', 'Казахстан')	Истина, если значение есть в списке
NOT IN	Проверяет, что значение не входит в список	country NOT IN ('Россия', 'Беларусь', 'Казахстан')	Истина, если значение не входит в список

3 Операторы объединения данных

3.1 Какие виды джойнов вы знаете?

1. INNER JOIN

Возвращает строки, имеющие соответствия в обеих таблицах, которые мы объединяем

2. LEFT JOIN (или LEFT OUTER JOIN)

Возвращает все строки из левой таблицы и соответствующие строки из правой таблицы. Если в правой таблице нет соответствия, то возвращается NULL.

3. RIGHT JOIN (или RIGHT OUTER JOIN)

Возвращает все строки из правой таблицы и соответствующие строки из левой таблицы. Если в левой таблице нет соответствия, то возвращается NULL.

4. FULL JOIN (или FULL OUTER JOIN)

Возвращает строки, имеющие соответствия в обеих таблицах, а также строки из обеих таблиц, где нет соответствий.

5. CROSS JOIN

Возвращает декартово произведение строк из двух таблиц, то есть все возможные комбинации строк из обеих таблиц.

3.2 В чем отличие между INNER JOIN и LEFT JOIN?

Оба оператора, **INNER JOIN** и **LEFT JOIN**, работают в два этапа:

1. **Этап декартова произведения (Cartesian Product):** Для каждой строки из левой таблицы система находит все строки из правой таблицы, где условие соединения (**ON ...**) выполняется. Важно понимать: если для одного ключа в левой таблице есть несколько подходящих строк в правой (или наоборот), произойдет "размножение" строк. Одна строка слева будет соединена с каждой подходящей строкой справа, что приведет к нескольким строкам в результате.
2. **Этап фильтрации:**
 - Для **INNER JOIN**: В результирующую выборку попадают только и только те строки, для которых условие в **ON** оказалось истинным (т.е. этап 1 дал хотя бы один результат для пары строк).
 - Для **LEFT JOIN**: В результирующую выборку попадают ВСЕ строки из левой таблицы. Для тех из них, для которых условие в **ON** оказалось истинным, добавляются данные из правой таблицы. Для тех, для которых условие не выполнилось, значения соответствующих колонок правой таблицы заполняются **NULL**.

Ключевое отличие, следовательно, не только в наличии **NULL**, а в гарантии сохранения всех строк из левой таблицы при использовании **LEFT JOIN**, независимо от наличия совпадений в правой. **INNER JOIN** такой гарантии не дает — он действует как фильтр, оставляя только строки с совпадениями в обеих таблицах.

3.3 В чем разница между UNION и UNION ALL?

Характеристика	UNION	UNION ALL
Обработка дубликатов	Удаляет повторяющиеся строки из финального результата	Сохраняет все строки, включая дубликаты
Производительность	Медленнее, так как требует дополнительного шага для сортировки и удаления дубликатов (дорогая операция)	Быстрее, так как просто склеивает все результаты без проверки
Использование	Когда необходимо уникальное множество записей	Когда нужны все данные из всех выборок, или когда вы уверены, что дубликатов нет

4 Оконные функции

4.1 Синтаксис оконных функций

Оконные функции в SQL всегда строятся по одному принципу: сначала указывается сама функция (например, SUM, AVG, ROW_NUMBER), потом через OVER() задаются условия, по которым эта функция будет работать.

Синтаксис оконной функции выглядит следующим образом:

```
1 SELECT <window_function>(arguments) OVER ([Partitioning] [Ordering]
   [Frame])
2 FROM table_name
```

Листинг 1: Общий синтаксис оконных функций

Компоненты:

- Имя оконной функции <window_function_name>
- Аргументы функции (arguments)
- Ключевое слово OVER()
- Разделение на партии [Partitioning]
- Сортировка [Ordering]
- Фрейм [Frame]

4.2 Какие виды оконных функций вы знаете?

Стоит вспомнить, что бывает всего 3 вида оконных функций:

1. Агрегирующие - позволяют вывести агрегат по окну рядом с детальными данными каждой строки

- SUM() — вычисляет сумму значений в окне
- AVG() — вычисляет среднее значение
- COUNT() — подсчитывает количество строк в окне
- MIN() / MAX() — находят минимальное и максимальное значение в окне

2. Ранжирующие - используются для присвоения порядковых номеров или рангов строкам внутри окна на основе заданного порядка (ORDER BY в рамках окна)

- ROW_NUMBER() — присваивает уникальный последовательный номер каждой строке в окне (1, 2, 3, ...). Даже если значения одинаковы, номер будет разным.
- RANK() — присваивает ранг строке. Если значения одинаковы, они получают одинаковый ранг, а следующий ранг пропускается (1, 2, 2, 4, ...).
- DENSE_RANK() — тоже присваивает одинаковый ранг одинаковым значениям, но следующий ранг не пропускается (1, 2, 2, 3, ...).
- NTILE(n) — разбивает все строки в окне на n примерно равных групп (квантилей) и присваивает каждой строке номер группы.

3. Значения - позволяют обращаться к данным из другой строки в том же окне (например, к предыдущей или следующей). Это мощный инструмент для анализа временных рядов и вычисления разниц.

- LAG(column, offset) — обращается к значению из строки, находящейся на offset строк выше (раньше) текущей.
- LEAD(column, offset) — обращается к значению из строки, находящейся на offset строк ниже (позже) текущей.
- FIRST_VALUE(column) — возвращает значение из первой строки в окне.
- LAST_VALUE(column) — возвращает значение из последней строки в окне. (Важно: по умолчанию учитывает рамки окна, часто требует указания ROWS BETWEEN CURRENT ROW AND UNBOUNDED FOLLOWING)

4.3 В чем разница между RANK() и DENSE_RANK()?

Обе функции используются для ранжирования строк внутри секции окна на основе заданного порядка (ORDER BY). Ключевое различие между ними заключается в том, как они обрабатывают ситуации с одинаковыми значениями (ничьими) и как это влияет на последующий ранг.

Функция RANK:

- Присваивает одинаковый ранг строкам с одинаковыми значениями.
- Пропускает следующие порядковые номера после каждого "пучка" одинаковых рангов.
- Нумерация становится непоследовательной (1, 2, 2, 4...).

Функция DENSE_RANK:

- Также присваивает одинаковый ранг строкам с одинаковыми значениями.
- НЕ пропускает следующие порядковые номера. Ранги идут по порядку.
- Нумерация остается последовательной (1, 2, 2, 3...)

4.4 Разница между LAG() и LEAD()?

Обе функции относятся к функциям смещения (сдвига) и позволяют обращаться к данным из других строк в том же результирующем наборе, не используя самосоединение (self-join). Ключевое различие между ними заключается в направлении, в котором они смотрят относительно текущей строки.

Функция LAG

- Позволяет обратиться к данным из предыдущей строки в окне.
- Смотрит назад (от англ. lag — отставание).
- Полезен для сравнения текущего значения с предыдущим (например, "сколько было вчера?").

Функция LEAD

- Позволяет обратиться к данным из следующей строки в окне.
- Смотрит вперед (от англ. lead — опережение).
- Полезен для сравнения текущего значения со следующим (например, "а что будет завтра?").

4.5 Фреймы: ROWS и RANGE. Что это такое и как они работают?

Фрейм — это "подвижное окно" из строк, которое "скользит" по данным и определяет, какие строки участвуют в расчете для каждой текущей строки. Теперь пришло время разобраться на примерах, как оно работает.

Синтаксис для определения границ фрейма:

```
1 ROWS BETWEEN UNBOUNDED PRECEDING AND CURRENT ROW
2 --
3 RANGE BETWEEN 2 PRECEDING AND 2 FOLLOWING
```

Листинг 2: Синтаксис определения фрейма

Границами могут служить любые из следующих пяти вариантов:

- UNBOUNDED PRECEDING — от начала раздела
- N PRECEDING — N строк/значений назад
- CURRENT ROW — текущая строка/значение
- N FOLLOWING — N строк/значений вперед
- UNBOUNDED FOLLOWING — до конца раздела

ROWS отбирает строки по их порядку — неважно, какие у них значения.

RANGE отбирает строки по логическому условию — если у нескольких строк значения попадают в логический диапазон, значит что по ним пройдет расчет оконной функции в едином окне.

5 Что такое подзапрос SQL?

5.1 Подзапрос SQL

«Подзапрос — это инструмент для вложения одного запроса в другой. Его основная цель — использовать результат внутреннего запроса для условий или вычислений во внешнем. Они бывают скалярные, возвращающие одно значение, и табличные. Также важно различать независимые подзапросы, которые выполняются один раз, и коррелированные, которые выполняются для каждой строки внешнего запроса и потому требуют внимания к производительности. Часто подзапросы можно переписать на JOIN, и наоборот, и выбор зависит от читаемости и эффективности в конкретной ситуации».

5.2 Как используется подзапрос?

Скалярный подзапрос

Возвращает одну строку и один столбец. Может использоваться практически в любом месте, где допустимо выражение: в SELECT, WHERE, HAVING.

```
1 SELECT name, salary
2 FROM employees
3 WHERE salary > (SELECT AVG(salary) FROM employees);
```

Листинг 3: Найти всех сотрудников с зарплатой выше средней

В условии WHERE (с операторами IN, ANY/ALL, EXISTS)

Возвращает набор значений (один столбец, много строк). Часто используется с оператором IN.

```
1 SELECT name
2 FROM customers
3 WHERE id IN (SELECT DISTINCT customer_id FROM orders);
```

Листинг 4: Найти всех клиентов

В качестве производной таблицы (в FROM)

Возвращает целую таблицу (несколько столбцов и строк). Должен иметь псевдоним.

```
1 SELECT AVG(department_avg)
2 FROM (
3     SELECT department_id, AVG(salary) AS department_avg
4     FROM employees
5     GROUP BY department_id
6 ) AS dept_avgs;
```

Листинг 5: Найти среднюю зарплату по отделам

6 Оптимизация кода

6.1 Как оптимизировать запрос?

Главный принцип: УМЕНЬШАЙ объем данных на КАЖДОМ этапе запроса.

1. Фильтруй РАНЬШЕ:

- WHERE перед GROUP BY и JOIN.
- Фильтруй таблицы перед их соединением.

2. Не ломай ИНДЕКСЫ:

- Не оборачивай поля в функции в условиях JOIN/WHERE (например, UPPER(column)).

3. Выбирай правильные ОПЕРАТОРЫ:

- EXISTS > IN для подзапросов (работает до первого совпадения).
- UNION ALL > UNION (если дубликаты не критичны).

4. Упрощай СТРУКТУРУ:

- Вместо вложенных подзапросов используй CTE (WITH). Код станет чище и может выполняться быстрее.

5. Избегай ИЗБЫТОЧНОСТИ:

- Не используй оконные функции (OVER()), если хватит простых (GROUP BY).
- Не используй ORDER BY без необходимости — это самая дорогая операция.

6.2 В чем отличие EXPLAIN от EXPLAIN ANALYZE?

EXPLAIN показывает предполагаемый план выполнения запроса без его реального выполнения. Можно увидеть:

- Порядок операций (сканирование, соединения, сортировка)
- Оценочную стоимость и количество строк
- Используются ли индексы

EXPLAIN ANALYZE фактически выполняет запрос и показывает реальную статистику:

- Фактическое время выполнения каждой операции
- Реальное количество обработанных строк
- Позволяет найти узкие места производительности

7 Общие вопросы про базы данных

7.1 Какие отличия между объектами БД Table, View, Materialized View, External Table?

Объект	Что это	Хранит данные?	Когда использовать	Подводные камни
Table	Основное место хранения данных (строки и колонки в СУБД)	Да	Для постоянного хранения «истины» (сырые и витринные данные)	Рост размера, блокировки при массовых обновлениях, необходимость в оптимальной стратегии хранения
View	Сохранённый SQL-запрос (виртуальная таблица)	Нет	Упрощение сложных запросов, безопасность (скрыть колонки), повторное использование логики	Выполняется при каждом обращении, нельзя индексировать, изменения в базовых таблицах могут сломать view
Materialized View	Сохранённый результат запроса (снимок данных)	Да	Ускорение тяжёлых агрегаций/джойнов, BI-дашборды, отчёты, промежуточные слои ETL	Может устаревать (требуется REFRESH), занимает место на диске, обновление может быть дорогим
External Table	Схема для доступа к данным вне СУБД (файлы, S3, другие БД)	Нет (данные вне базы)	Быстрый доступ к сырым данным без загрузки, интеграция с другими источниками (ETL процесс)	Зависимость от сети, медленные запросы, ограничения фильтрации/индексации, риск несоответствия схемы

7.2 Что такое транзакция?

Транзакция — это неделимый блок из нескольких операций в базе данных, который должен быть выполнен либо целиком, либо не выполнен вообще.

Главное правило транзакций — «всё или ничего». Все действия внутри нее должны быть завершены успешно. Если хотя бы одна операция провалится, отменятся все изменения, сделанные в этой транзакции. Классический пример — тот самый банковский перевод. Транзакция гарантирует, что если мы списали X рублей с одного счета, то ровно X рублей и будут зачислены на другой. Баланс системы никогда не нарушится.

7.3 Принципы ACID

ACID - это набор принципов, который определяет как база данных должна обрабатывать операции, чтобы данные оставались корректными в любых обстоятельствах

- A — Atomicity (Атомарность)
- C — Consistency (Согласованность)
- I — Isolation (Изолированность)
- D — Durability (Надёжность)

7.4 Индексы БД. Что это такое?

Индексы — это объекты БД, ускоряющие процессы чтения данных (фильтрации, поиск и тд). Без индексов СУБД просматривает таблицу построчно, как если бы вы искали слово в словаре, перелистывая все страницы подряд, до тех пор, пока не наткнулись на него. Процесс полного построчного чтения называется Full Table Scan и является довольно калорийной операцией, которую по возможности следует избегать.

Как раз для этого и придумали индексы, ведь они работают по принципу оглавления книги — внутри индекса хранятся упорядоченные значения и ссылки на строки, где эти данные находятся. Благодаря этому база может сразу перейти к нужному месту, не просматривая всю таблицу. Грубо говоря, если нам попало слово на букву "Я мы сразу перейдем на страницу, с которой начинается этот раздел и будем искать только среди этих слов, минуя остальные разделы.

Таким образом, роль индексов состоит в том, чтобы облегчить поиск подмножества строк и столбцов таблицы без необходимости сканировать каждую строку в таблице.